

Informe de Laboratorio 06

Tema: Django Admin y Modelos

Nota

Integrantes (Grupo 2)	Escuela	Asignatura
Salas Zegarra, Marco Antonio Molina Soncco, Percy Vicente Medina, Eder Lucio Peralta Llasa, Kevin	Escuela Profesional de Ingeniería de Sistemas	Desarrollo de Aplicaciones Web Semestre: I Codigo: 1703122

Laboratorio	Tema	Duración
06	Django Admin y Modelos	06 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - A	18 de mayo 2026	24 de mayo 2026

1. Objetivos del Laboratorio

- Mostrar el modelo de datos resumido para el proyecto final.
- Crear un proyecto Django y una aplicación específica (**sissub**).
- Crear modelos en archivos independientes aplicando restricciones desde el modelo.
- Redefinir los métodos `def save()` y `def __str__()` para personalizar el comportamiento y la visualización de los registros.
- Generar e implementar auto CRUDs usando Django Admin.

2. Desarrollo Técnico

2.1. Estructura del Proyecto y Modelos Independientes

El entorno de desarrollo se organizó respetando el patrón arquitectónico de Django, aislando los modelos en un paquete `models` dentro de la aplicación **sissub**. A continuación se presenta la estructura del directorio raíz **backend**:

Listing 1: Estructura de directorios del Backend

```
backend/  
|-- mgs/                # Proyecto principal de Django
```

```
| |--- settings.py
| |--- urls.py
| |--- wsgi.py
|--- sissub/                # Aplicacion principal
| |--- migrations/
| |--- models/             # Modelos desacoplados en archivos
| | |--- __init__.py
| | |--- category.py
| | |--- currency.py
| | |--- service.py
| | |--- subscription_user.py
| | |--- subscription.py
| | |--- user_profile.py
| |--- admin.py
| |--- apps.py
| |--- views.py
|--- .env                  # Variables de entorno
|--- .gitignore
|--- db.sqlite3
|--- diagrama_der.png
|--- manage.py
|--- requirements.txt
```

2.2. Creacion de Modelos y Restricciones

Se implementaron restricciones personalizadas empleando la librería `ValidationError` de `django.core.exceptions`. Asimismo, se modificó el comportamiento nativo de los modelos redefiniendo los métodos `__str__()` y `save()`.

2.2.1. Modelo Service (service.py)

Se validó que el nombre de cada servicio posea un mínimo de caracteres y se aplicó el formateo automático previo al guardado.

Listing 2: Código fuente del modelo Service

```
from django.db import models
from django.core.exceptions import ValidationError

def validar_nombre(value):
    if len(value.strip()) < 3:
        raise ValidationError("El nombre debe contener al menos 3 caracteres.")

class Service(models.Model):
    id = models.AutoField(primary_key=True)
    name = models.CharField(max_length=100, validators=[validar_nombre])
    is_custom = models.BooleanField(default=False, db_column='iscustom')
    status = models.CharField(max_length=20, default='Active')

    created = models.DateTimeField(auto_now_add=True)
    modified = models.DateTimeField(auto_now=True)
    created_id = models.UUIDField(null=True, blank=True)
    modified_id = models.UUIDField(null=True, blank=True)

    class Meta:
```

```
managed = False
db_table = 'services'

def __str__(self):
    return f"Servicio: {self.name} [{self.status}]"

def save(self, *args, **kwargs):
    if self.name:
        self.name = self.name.strip()

    self.full_clean()
    super().save(*args, **kwargs)
```

2.2.2. Modelo Subscription (subscription.py)

Este modelo integra múltiples claves foráneas y restricciones avanzadas, como evitar que las suscripciones posean un monto nulo/negativo o fechas de facturación expiradas.

Listing 3: Código fuente del modelo Subscription

```
from django.db import models
from django.core.exceptions import ValidationError
from datetime import date

def validar_monto_positivo(value):
    if value <= 0:
        raise ValidationError("El monto debe ser estrictamente mayor a cero.")

def validar_fecha_futura(value):
    if value and value < date.today():
        raise ValidationError("La fecha de facturación no puede ser pasada.")

class Subscription(models.Model):
    id = models.AutoField(primary_key=True)
    user = models.ForeignKey('sisub.UserProfile', on_delete=models.CASCADE,
                             db_column='user_id')
    service = models.ForeignKey('sisub.Service', on_delete=models.DO_NOTHING,
                                db_column='service_id')
    category = models.ForeignKey('sisub.Category', on_delete=models.SET_NULL, null=True,
                                 blank=True, db_column='category_id')
    currency = models.ForeignKey('sisub.Currency', on_delete=models.DO_NOTHING,
                                 db_column='currency_id')

    amount = models.DecimalField(max_digits=10, decimal_places=2,
                                 validators=[validar_monto_positivo])
    billing_cycle = models.CharField(max_length=50, db_column='billingcycle')
    next_billing_date = models.DateField(db_column='nextbillingdate',
                                         validators=[validar_fecha_futura])
    status = models.CharField(max_length=20, default='Active')

    created = models.DateTimeField(auto_now_add=True)
    modified = models.DateTimeField(auto_now=True)
    created_id = models.UUIDField(null=True, blank=True)
    modified_id = models.UUIDField(null=True, blank=True)

    class Meta:
```

```
managed = False
db_table = 'subscriptions'

def __str__(self):
    return f"Suscripcion {self.id}: {self.service.name} - {self.amount} {self.currency.id}"

def save(self, *args, **kwargs):
    if self.billing_cycle:
        self.billing_cycle = self.billing_cycle.lower().strip()
    self.full_clean()
    super().save(*args, **kwargs)
```

2.3. Configuración del Django Admin (admin.py)

Para dotar al sistema de auto CRUDs efectivos, se registraron los modelos en el archivo administrativo, personalizando columnas de visualización, filtros y barras de búsqueda.

Listing 4: Código fuente del panel administrativo (admin.py)

```
from django.contrib import admin
from .models import UserProfile, Currency, Service, Category, Subscription, SubscriptionUser

@admin.register(UserProfile)
class UserAdmin(admin.ModelAdmin):
    list_display = ('id', 'email', 'username', 'status', 'created')
    search_fields = ('email', 'username')
    list_filter = ('status',)

@admin.register(Currency)
class CurrencyAdmin(admin.ModelAdmin):
    list_display = ('id', 'symbol', 'status')
    search_fields = ('id',)

@admin.register(Service)
class ServiceAdmin(admin.ModelAdmin):
    list_display = ('id', 'name', 'is_custom', 'status')
    list_filter = ('is_custom', 'status')
    search_fields = ('name',)

@admin.register(Category)
class CategoryAdmin(admin.ModelAdmin):
    list_display = ('id', 'name', 'user', 'status')
    search_fields = ('name',)

@admin.register(Subscription)
class SubscriptionAdmin(admin.ModelAdmin):
    list_display = ('id', 'user', 'service', 'amount', 'currency', 'billing_cycle',
                    'next_billing_date', 'status')
    list_filter = ('status', 'billing_cycle', 'currency')
    date_hierarchy = 'next_billing_date'

@admin.register(SubscriptionUser)
class SubscriptionUserAdmin(admin.ModelAdmin):
    list_display = ('id', 'subscription', 'user', 'percentage', 'status')
    list_filter = ('status',)
```

2.4. Diagrama Entidad-Relación (DER) en Django

Para la generación del diagrama DER, se utilizó la librería `django-extensions` y la herramienta `Graphviz`. Mediante el comando `python manage.py graph_models sissub -o diagrama_der.png`, se exportó automáticamente el modelo relacional configurado en la aplicación.

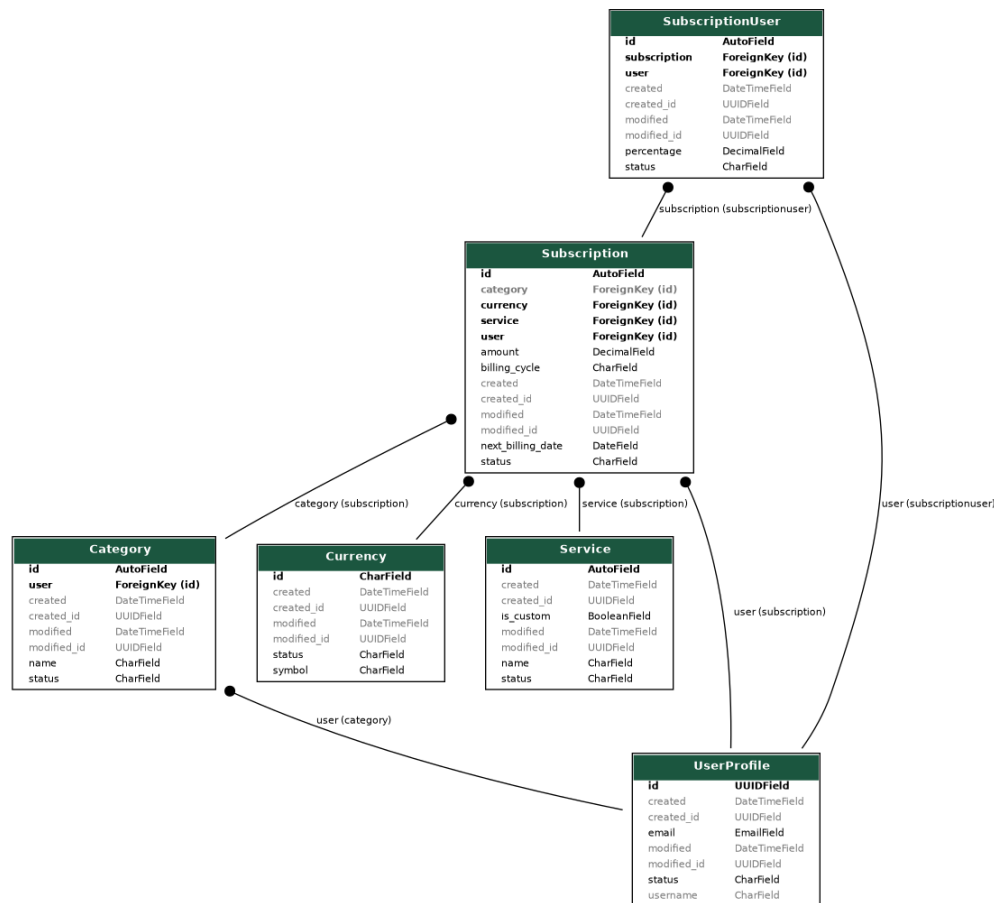


Figura 1: Diagrama Entidad-Relación generado por django-extensions.

2.4.1. Operaciones CRUD

Para demostrar el funcionamiento del Auto CRUD generado por Django Admin y la correcta aplicación de las restricciones a nivel de base de datos, se documenta el proceso de creación de un nuevo registro para la entidad **Service**.

Paso 1: Acceso al formulario de creación

Desde el panel principal (Dashboard) del administrador, se selecciona el botón **+ Add** correspondiente al modelo *Services* dentro de la aplicación *SISSUB*. Esto despliega una interfaz autogenerada por Django, cuyos campos de entrada (*inputs*) se adaptan automáticamente a los tipos de datos definidos en `service.py`.

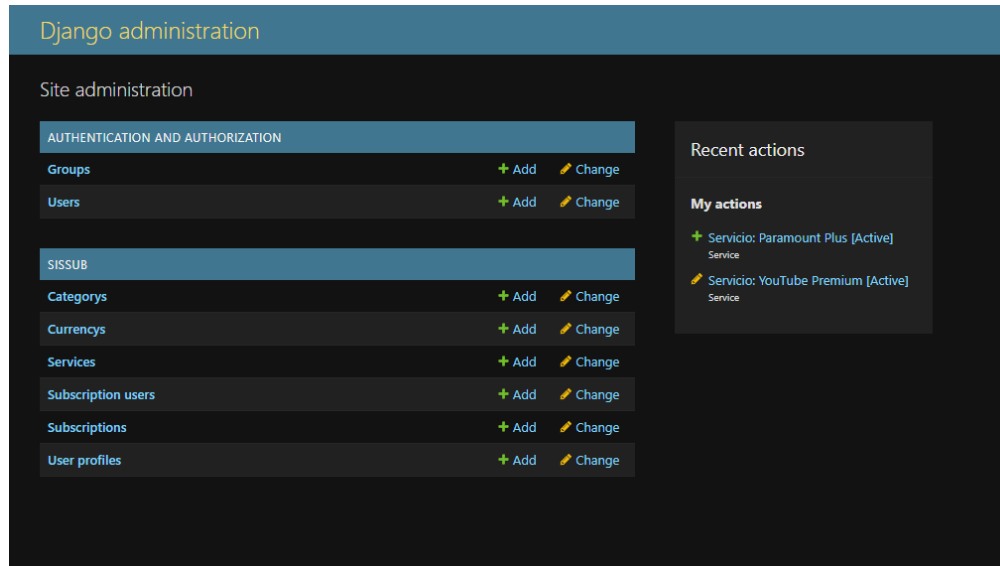


Figura 2: Interfaz del panel de administrador de Django.

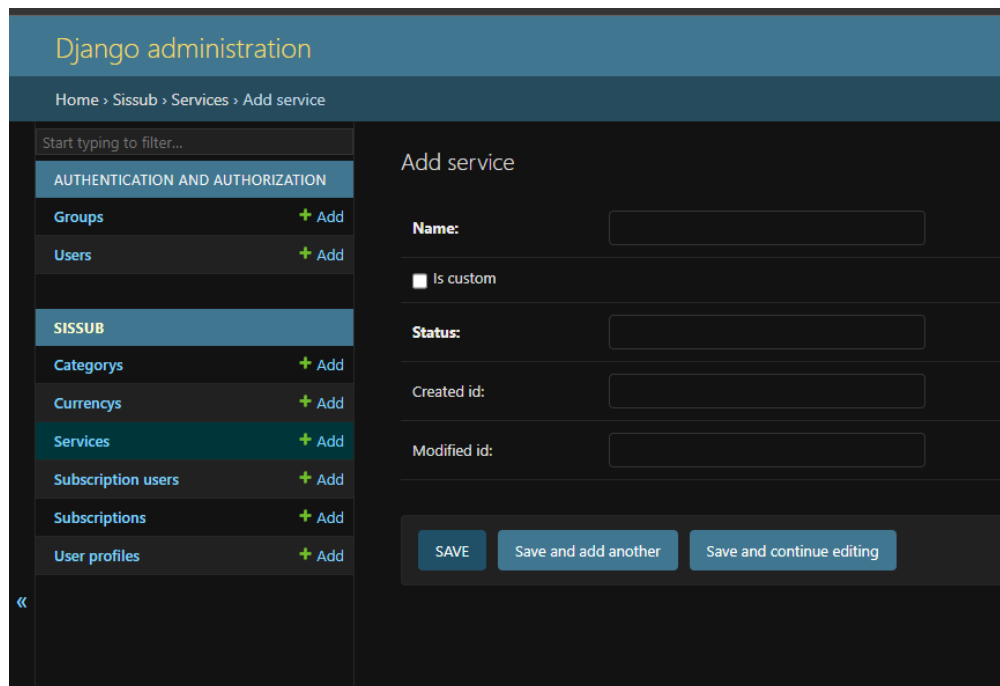
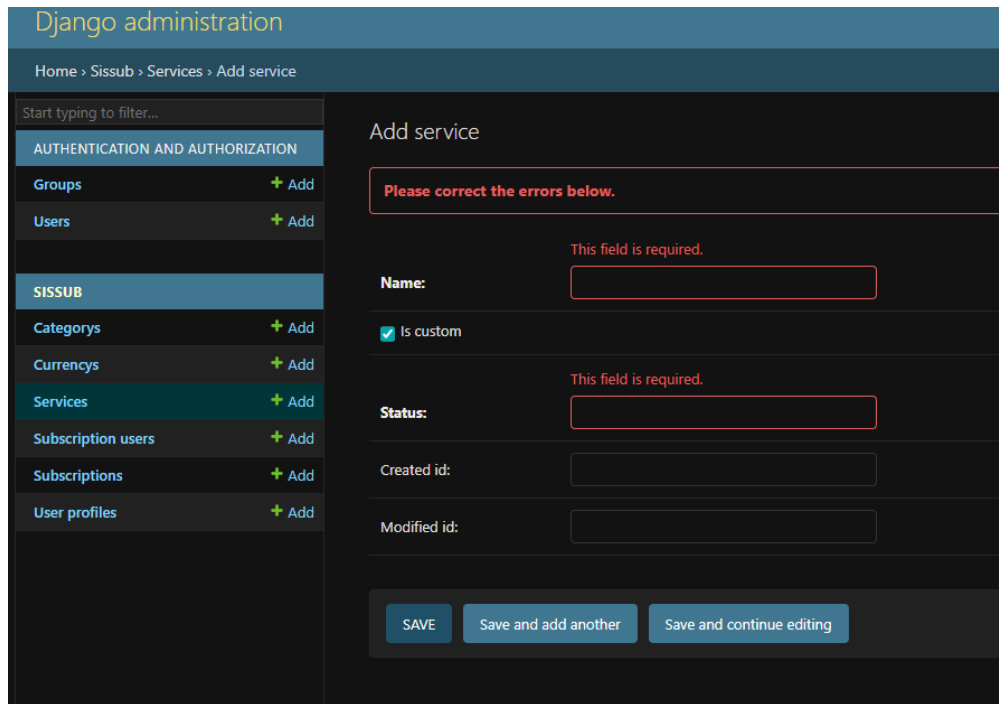


Figura 3: Interfaz autogenerada para la creación de un nuevo servicio.

Paso 2: Ingreso de datos e interacción con validadores

Se completan los campos solicitados por el sistema. Es fundamental destacar que el campo **name** está protegido por el método **validar_nombre**, el cual impide el registro si la cadena tiene menos de 3 caracteres. Por otro lado, el campo **is_custom** se renderiza de forma nativa como un *checkbox* debido a su naturaleza booleana, y **status** carga su valor por defecto (**Active**).



Django administration

Home > Sissub > Services > Add service

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

- Groups + Add
- Users + Add

SISSUB

- Categorys + Add
- Currencys + Add
- Services + Add
- Subscription users + Add
- Subscriptions + Add
- User profiles + Add

Add service

Please correct the errors below.

Name: This field is required.

☒ Is custom

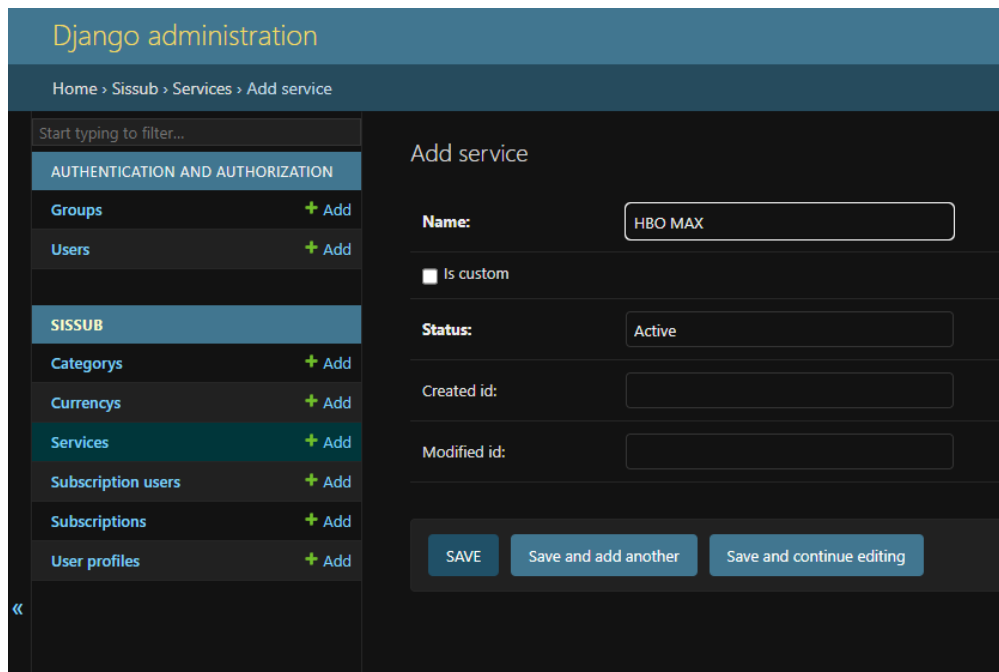
Status: This field is required.

Created id:

Modified id:

SAVE Save and add another Save and continue editing

Figura 4: Verificación de algunas restricciones.



Django administration

Home > Sissub > Services > Add service

Start typing to filter...

AUTHENTICATION AND AUTHORIZATION

- Groups + Add
- Users + Add

SISSUB

- Categorys + Add
- Currencys + Add
- Services + Add
- Subscription users + Add
- Subscriptions + Add
- User profiles + Add

Add service

Name:

☐ Is custom

Status:

Created id:

Modified id:

SAVE Save and add another Save and continue editing

Figura 5: Ingreso de datos respetando las restricciones del modelo.

Paso 3: Persistencia y ejecución del método save()

Al presionar el botón **Save**, Django procesa la petición y ejecuta el método `save()` redefinido en el

modelo. Este método aplica la función `strip()` para eliminar espacios en blanco accidentales al inicio o final del nombre. Una vez superada la validación con `full_clean()`, el registro se inserta en Supabase y la interfaz retorna un mensaje de confirmación verde.

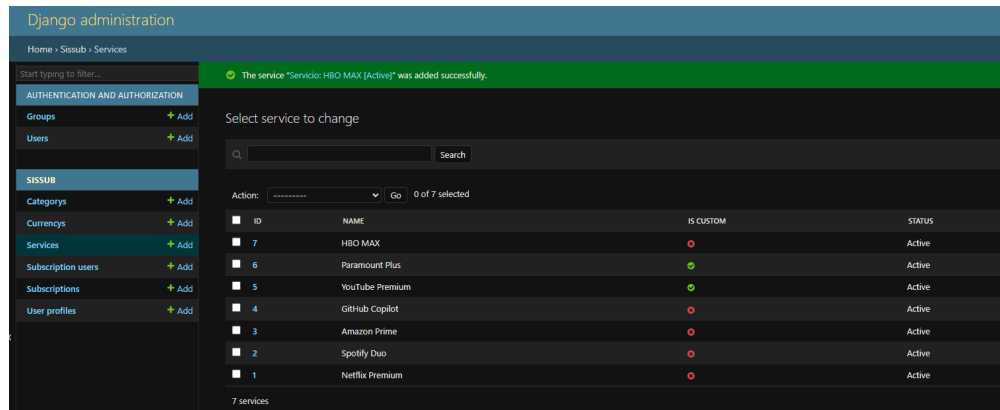
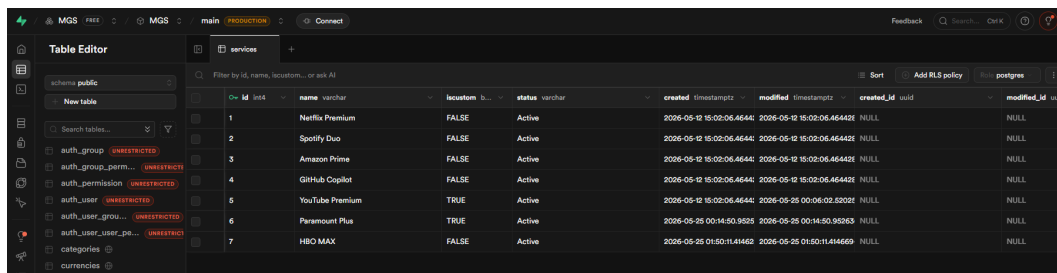


Figura 6: Confirmación de guardado exitoso y visualización del nuevo registro.



id	name	iscustom	status	created	modified	created_id	modified_id
1	Netflix Premium	FALSE	Active	2026-05-12 15:02:06.4644	2026-05-12 15:02:06.46442	NULL	NULL
2	Spotify Duo	FALSE	Active	2026-05-12 15:02:06.4644	2026-05-12 15:02:06.46442	NULL	NULL
3	Amazon Prime	FALSE	Active	2026-05-12 15:02:06.4644	2026-05-12 15:02:06.46442	NULL	NULL
4	GitHub Copilot	FALSE	Active	2026-05-12 15:02:06.4644	2026-05-12 15:02:06.46442	NULL	NULL
5	YouTube Premium	TRUE	Active	2026-05-12 15:02:06.4644	2026-05-25 00:06:02.8202	NULL	NULL
6	Paramount Plus	TRUE	Active	2026-05-25 00:14:50.9825	2026-05-25 00:14:50.98253	NULL	NULL
7	HBO MAX	FALSE	Active	2026-05-25 01:50:11.4462	2026-05-25 01:50:11.44669	NULL	NULL

Figura 7: Actualización de los registros en el Supabase.

2.5. Proceso de Implementación y Evolución del Proyecto

Para garantizar un desarrollo ordenado y escalable, la construcción del proyecto siguió un flujo de trabajo estructurado. A continuación, se detallan los pasos clave y los comandos ejecutados en la terminal para levantar el entorno y configurar el administrador de Django.

Fase 1: Preparación del Entorno y Dependencias

El primer paso consistió en aislar el proyecto utilizando un entorno virtual para evitar conflictos de versiones. Posteriormente, se instalaron los paquetes requeridos, incluyendo `django-enviro` para la gestión segura de variables de entorno y `psycopg2-binary` para la conexión a PostgreSQL.

Listing 5: Comandos de inicialización del entorno

```
# Activación del entorno virtual (WSL/Ubuntu)
$ source venv/bin/activate

# Instalación de dependencias del proyecto
$ pip install -r requirements.txt
```

Fase 2: Aislamiento de Credenciales

En lugar de exponer la base de datos en el código fuente, se implementó el uso de un archivo `.env`

local. En el archivo `settings.py`, se configuró la lectura de estas variables para conectar el proyecto a la instancia de Supabase en la nube.

Listing 6: Código clave en `settings.py` para la conexión segura

```
import environ

env = environ.Env()
environ.Env.read_env() # Lee el archivo .env local

DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': env('DB_NAME'),
        'USER': env('DB_USER'),
        'PASSWORD': env('DB_PASSWORD'),
        'HOST': env('DB_HOST'),
        'PORT': env('DB_PORT'),
    }
}
```

Fase 3: Migraciones de los Modelos

Una vez definidos los modelos independientes en la aplicación `sissub`, se procedió a generar los archivos de migración y a sincronizarlos con la base de datos PostgreSQL en Supabase.

Listing 7: Ejecución de migraciones hacia la base de datos

```
# Generacion de las instrucciones SQL a partir de los modelos
$ python manage.py makemigrations sissub

# Aplicacion de los cambios en la base de datos en la nube
$ python manage.py migrate
```

Fase 4: Creación del Administrador

Finalmente, para poder visualizar e interactuar con los modelos a través de la interfaz gráfica, se creó un superusuario con privilegios administrativos y se levantó el servidor de desarrollo local.

Listing 8: Creación del superusuario y ejecución del servidor

```
# Creacion de la cuenta de administrador
$ python manage.py createsuperuser

# Ejecucion del servidor de desarrollo
$ python manage.py runserver

# Servidor corriendo en : http://127.0.0.1:8000/
```

2.6. Control de Versiones y Trabajo Colaborativo

El desarrollo incluyó una gestión de versiones estricta en GitHub, destacando la configuración del entorno y la creación de los modelos esenciales.

Listing 9: Secuencia de comandos Git para el trabajo colaborativo

```
$ git add .
$ git commit -m "Crea todos los modelos esenciales"
```

```
$ git add .
$ git commit -m "chore: agregar exclusiones de seguridad para entorno local"

$ git add .
$ git commit -m "build: aislar credenciales mediante variables de entorno y fijar
dependencias"

$ git add .
$ git commit -m "Crea un user admin"

$ git add .
$ git commit -m "Merge pull request #4 from evicente3/msalasz"
```

2.7. Resumen de Accesos y Evidencias Digitales

Con el objetivo de facilitar la revisión y validación de los entregables desarrollados durante la práctica de laboratorio, se presentan las principales evidencias digitales relacionadas con el proyecto. Estas evidencias incluyen el repositorio grupal en GitHub, donde se almacenaron los archivos del sistema y la documentación del proyecto, así como la implementación de la base de datos en Supabase mediante la importación del script SQL desarrollado en PostgreSQL.

Las capturas presentadas permiten verificar la estructura general del repositorio, la organización de los módulos del proyecto y la correcta creación de las tablas y relaciones dentro del entorno cloud proporcionado por Supabase.

1. **Repositorio en GitHub (Código Fuente y Versionado):** Espacio de trabajo colaborativo donde se gestionó el historial de commits y la integración de las ramas.

<https://github.com/evicente3/MGS/tree/main/backend>

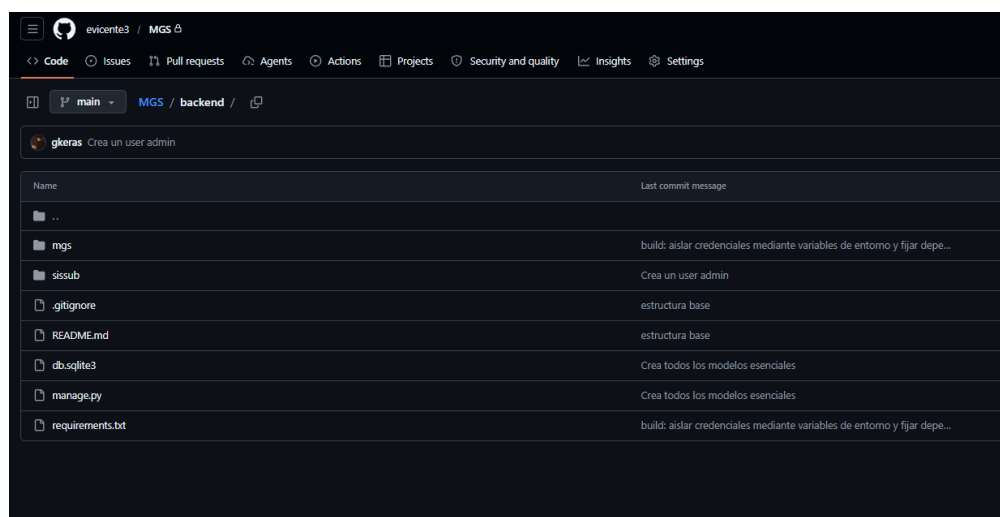


Figura 8: Captura de pantalla del repositorio oficial en GitHub.

2. **Base de Datos en Supabase:** Visualización de las tablas y relaciones implementadas en Supabase mediante la herramienta *Schema Visualizer*, verificando la correcta importación del modelo físico desarrollado en PostgreSQL.

<https://supabase.com/dashboard/project/zdjppofxtnfbjagjhbia/database/schemas>

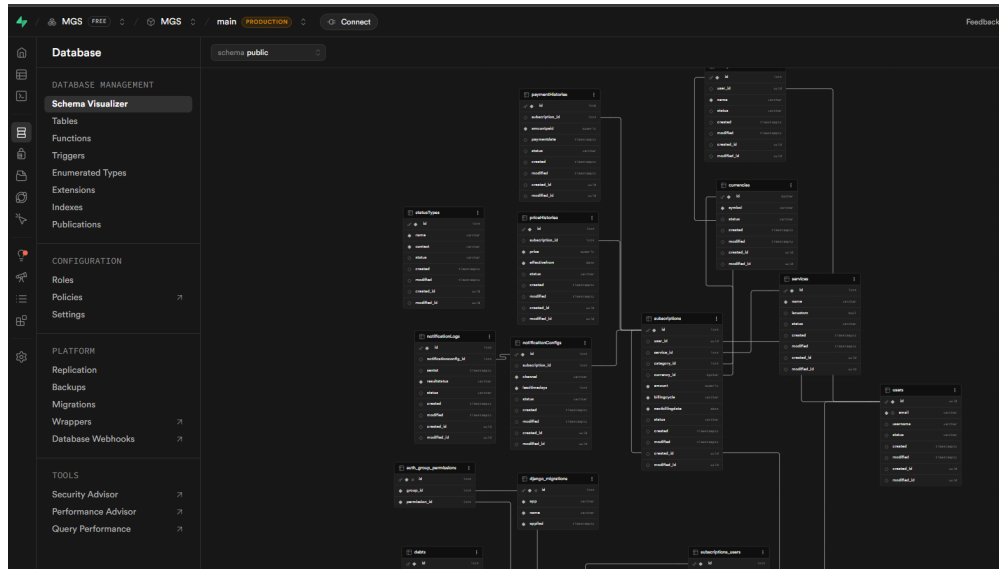


Figura 9: Captura de pantalla de la base de datos implementada en Supabase.

3. Conclusiones

- La separación de modelos en archivos independientes mejora la escalabilidad del código fuente en proyectos Django, facilitando el mantenimiento y la delegación de responsabilidades en equipos de desarrollo.
- La redefinición de los métodos `save()` y `__str__()` otorga control directo sobre la lógica de negocio (ej. limpieza de espacios en blanco y minúsculas automáticas) previa a la persistencia.
- La implementación de validadores a nivel de modelo garantiza la integridad de la información antes de que interactúe con el motor de base de datos PostgreSQL, evitando montos negativos y fechas caducadas.
- El uso de `django-extensions` automatizó exitosamente la documentación de la arquitectura de datos, validando visualmente las relaciones configuradas mediante llaves foráneas.

4. Rúbrica de Calificación

A continuación se detalla la rúbrica oficial de evaluación asignada para este laboratorio, la cual establece los criterios e indicadores técnicos junto con su respectiva ponderación para la nota final:

Cuadro 1: Rúbrica de calificación del Laboratorio 06.

Ítem	Descripción	Puntaje
Entorno Virtual	Uso correcto del entorno virtual en proyectos de Python.	2
Step 2: Django Admin	Realiza detalladamente los pasos para crear el administrador de Django a partir de los modelos.	3
Restricciones	Crear restricciones de forma asertiva a partir de los modelos de Django.	3
Modelo DER	Utiliza una herramienta para crear el modelo entidad-relación en Django.	3
BD	Envía la base de datos con los registros existentes para su revisión.	3
Modelos	Realiza las configuraciones necesarias en los modelos para cumplir con los estándares y obtener las funcionalidades solicitadas.	3
Informe	El laboratorio tiene un README.md que detalla toda la práctica.	3
Prueba	Se tomaron en cuenta todas las consideraciones y recomendaciones, lo que evidencia un trabajo en equipo.	-0
Total		18